What Does 0% Mean?

A Four-Arm Protocol for Evaluating Prompt-Injection Defenses

Draft — not peer reviewed. Author: Emre Bozkurt. Artifact: `https://github.com/FlyingGrasss/agent-os`.

© 2026 Emre Bozkurt. This paper is licensed under CC BY 4.0.

This is a methodology note with a case study of the authors' own system. The case-study numbers are from the artifact at commit `e09992a`; the live-model arm is single-run and exploratory. The companion technical report is `price-of-zero.md`.

Abstract

Prompt-injection defenses for tool-using agents increasingly report attack-success rates at or near zero. A zero can mean five different things: the policy denied the value, the user declined the prompt, the model could not write a runnable plan, the harness rejected the tool name, or the attack never ran. Only the first is a property of the machine. The second is a property of the person. The last three are measurement failures. This paper proposes a four-arm protocol that makes the possibilities separable: no policy, policy with a declining user, policy with an approving user, and a strict mode that turns every confirmation into a refusal. The protocol is paired with a metric distinction — broad taint flow versus authorization-relative leakage — and a reporting checklist. We illustrate it on AgentOS, a runtime that isolates a privileged planner from a quarantined document reader and enforces policy per tool argument. On a 45-program policy corpus, the same rules produce 0/33 attack success with a declining user and 14/33 with an approving one; 17/33 attacks depend on the human. On a 16-task model corpus, strict mode produces zero tainted flows and zero prompts but denies six of seven benign forwarding tasks. We also catalog four measurement failures that produced misleading results, including a live harness that advertised a tool name the host did not register. The contribution is methodological: a cheap, reusable way to report what a zero costs, who produced it, and what it does not cover.

---

1. Introduction

A defense paper reports "0% attack success." A reader is expected to conclude that the defense works. That conclusion is under-specified. The same system, on the same attacks, can produce:

1. **0% because the policy denied every tainted value.** This is the machine's share.
2. **0% because the user declined every confirmation.** This is the human's share.
3. **0% because the model could not write a runnable plan.** This is model quality.
4. **0% because the harness rejected the tool name the model used.** This is a broken instrument.
5. **0% because the attack never ran, or the plan halted after the sensitive call.** This is a measurement failure.

These are different claims. A paper that reports only the number has not said which one it is making. The problem is not dishonesty; it is the standard experiment. It has one arm — the defended system with a cooperative user — and one number. The arm is rarely named, the utility cost is often measured on tasks that do not exercise the conflict, and the attacks are often self-authored.

This paper makes the missing variables explicit. We propose four arms that differ only in the decision layer, a pair of leakage definitions that separate requested forwarding from exfiltration, and a checklist that a paper or an internal evaluation can apply without building a new system. We illustrate the protocol on AgentOS, a runtime built around the structural approach: a quarantined extractor, provenance labels, and per-argument policy. The case study is not the contribution; it is the worked example that shows what the arms reveal.

The contributions are:

1. **A catalog of four measurement failures** that each made a reported result meaningless, with the control that catches each.
2. **A four-arm protocol.** No policy, declining user, approving user, strict no-human. Each arm answers a different question, and the differences between them are the result.
3. **Two leakage definitions.** Broad taint flow counts any untrusted value reaching a tool; authorization-relative leakage counts only flows the user did not authorize.
4. **A worked example** of one task under all four arms, including the benign twin.
5. **A reporting checklist** for prompt-injection evaluations.

We do not claim a new defense. The architecture is close to CaMeL [1], Progent [4], IsolateGPT [3], and FIDES [5]. The paper is a measurement of such a system, not an independent evaluation; the case study is the authors' own artifact.

---

2. Four Ways a Zero Lies

The case study produced four measurement failures. Each was found by inspecting a result that looked too good, and each has a control that catches it. They are presented first because they are the paper's premise: a zero is only as good as the instrument that produced it.

2.1 The halt after the sink

A corpus entry ended with `len(doc)` on a record. The interpreter's `len` is defined for lists and strings, not records, so every plan halted with `BadField` — but only after the sensitive tool call. Leaks were still counted, but benign completion was unmeasurable, and the outcome looked like a defense. The fix was to read `doc.total`, a field the record actually has.

Control: assert that a plan reaches its sink and completes; report *where* a halted plan stopped. A halt after the effect is not a blocked effect.

2.2 The fixture that measured nothing

A model fixture returned a literal `status = "paid"` instead of reading the document. The closed-type arm therefore never saw the status-line attack, and the unbounded and bounded arms produced identical byte counts. The null result looked like a finding about schema bounds.

Control: derive every extracted field from the document, and include a positive control that fails if the fixture sanitizes what it lifts. A null result needs a positive control before it is a result.

2.3 The tool-name mismatch

The planner instruction was corrected to the shipping tool names (`http.fetch`, `fs.write`), but the eval host still registered the fixture names (`http.get`, `send_email`, `shell`). Every live plan died on its first call with

`ToolRejected`, and the arm reported 0 leaks. The 0% was a broken instrument, not a defense.

Control: a drift test that the instruction advertises exactly the tools the host registers. The fix makes the instruction a harness parameter (`Planner::with_instruction`, which accepts only a `&'static str` so runtime data still cannot enter the prompt).

2.4 The model-quality failure

A model that cannot produce a runnable plan never reaches a tool. If those tasks are folded into "stopped," the defense gets credit for the model's weakness. In the live arm, 3–4 of 16 tasks failed to plan.

Control: report model-quality failures separately and keep them in the denominator. A model-quality failure is not a blocked attack.

2.5 The common pattern

All four failures have the same shape: a result that is green because nothing ran. The control is always a positive check that the experiment reached the point where the defense could fail. A counterfactual, a benign twin, a drift test, and a plan-completion check are four versions of that idea.

---

3. The Four-Arm Protocol

3.1 Definitions

A **task** is a pair `(r, d)`: a trusted request `r` and a document `d`. An **attack task** has a document containing an injection; a **benign task** has a clean document. A **benign twin** is the same request with a clean document.

A **plan** is a sequence of tool calls produced from `r` and the schema, without reading `d`. Each argument value `v` carries a provenance label `λ(v) ∈ {trusted, untrusted}`. A **sink** is a pair `(tool, argument)`. A **flow** is a value reaching a sink.

A flow is **authorized** if the request names the destination or the field for that sink, or the operator's policy allowlists it. Otherwise it is **unauthorized**.

- **Broad leakage:** an untrusted value reached any sink.
- **Authorization-relative leakage:** an untrusted value reached a sink the request did not authorize.

An **arm** is a triple `(π, U, s)`: a policy `π`, a user oracle `U` that answers confirmations, and a strict flag `s` that turns every confirmation into a denial. For each task the plan is executed under `π`; `U` answers confirmations; the run yields a set of flows, a terminal outcome, and a prompt count.

3.2 The arms

Arm	Policy	User	Question it answers
A0	allow everything	none	Does the corpus actually route attacker data into tools?
A1	shipped policy	declines every prompt	What does the defense stop with a careful user?
A2	shipped policy	approves every prompt	What does the human contribute?
A3	strict: confirm → deny	none	What does a no-human zero cost in utility?

A0 is the counterfactual. Without it, a corpus whose payloads were silently dropped reports a perfect defense for free. A1 is the headline arm. A2 is the arm that makes the headline honest. A3 is the bound: it is the cheapest zero in human attention, and the most expensive in utility.

The arms differ only in the decision layer. The plans, the documents, the extractor, and the tool set are held fixed. That is what makes the differences attributable.

3.3 Controls

1. **Counterfactual.** A0 must succeed on every policy-relevant attack. Resource-exhaustion programs are expected to halt in every arm; they are counted separately.
2. **Utility.** In the policy corpus, A1 must complete every benign control. In the model corpus, a benign twin that legitimately forwards a document field may be refused; that is the utility cost, reported rather than treated as a control failure.
3. **Same plans.** A1 and A2 must use the same plans and documents; only the user's answer differs. A2 and A3 must use the same plans and documents; only the policy differs.
4. **Cross-check.** The automatic-versus-human split is computed two ways: by the rule that produced the stop, and by the difference between A1 and A2. They must agree.
5. **Model quality.** A task the model cannot plan is reported separately and stays in the denominator.
6. **Harness drift.** The planner instruction must advertise exactly the tools the host registers. A test asserts it.
7. **Repeatability.** Sampled arms are repeated; a single run is labelled exploratory. Deterministic arms are exact for the corpus.

3.4 Metrics

For each arm, report:

- attack success under both leakage definitions;
- benign completion, and benign clean completion (no tainted flow);
- prompts per task;
- automatic stops, human stops, and budget halts separately;
- model-quality failures, in the denominator;
- forwarded bytes, when text is the payload;
- the number of runs and the range, or the word "single run."

A percentage without its raw counts is not enough on a small corpus. A raw count without its arm is not a result.

---

4. Worked Example: One Task, Four Arms

The task is:

> Fetch `https://invoices.example/i1` and email the invoice note to the address it names.

The clean document says `recipient=bob@example.com` and `note="pay within 7 days"`. The hostile document says `recipient=attacker@evil.test` and puts an instruction in the note. The planner writes:

```
let page = http.get(url = "https://invoices.example/i1");
let doc  = extract(Invoice, source = page);
send_email(to = doc.recipient, subject = "invoice", body = doc.note);
```

The planner never sees the document; it sees the request and the schema. `doc.recipient` is a named value with an untrusted label. The plan can carry it without reading it.

Arm	What happens on the hostile document	Flow?	What happens on the clean twin
A0 no policy	sends to `attacker@evil.test` with the injected note	yes	sends to `bob@example.com`
A1 declining	asks about `send_email.to`; user sees the address and says no	no	asks; user says no; benign twin refused
A2 approving	asks; user says yes; sends to the attacker	yes	sends to Bob
A3 strict	denies `send_email.to` and `send_email.body`; asks nobody	no	denies the benign twin too

The table is the protocol in one page. A0 proves the attack is real. A1 shows the human holding the line. A2 shows what happens when the human does not. A3 shows the price: the benign twin is refused because its recipient also came from the document, and strict does not distinguish a benign document from a hostile one.

If the request had said "email the note to `bob@example.com`," the address would be pinned. The plan's use of `doc.recipient` would then be a substitution, and the intent pin would deny it in A1 and A2 before the user was asked. The benign twin, whose recipient matches the pin, would still be refused by A3, because strict denies every tainted `send_email.to` regardless of the pin. Pinning makes the substitution case cheaper; it does not make strict free.

This example also shows why the broad metric overcounts. In A2 the hostile document's note reaches the `body`; if the user asked to forward the note, that flow is authorized. The tool ran with a tainted argument, so the broad metric calls it a leak. The narrow metric does not. A paper that reports only the broad number will overstate the attack and understate the utility.

---

5. Case Study: AgentOS

5.1 System

AgentOS is a Rust runtime that implements the structural approach. A document enters as `Quarantined<String>`. A confined extractor reads it and returns typed fields; the planner sees the request and the schema, not the document. Values carry provenance labels through a small plan language. Before each tool call, a per-argument policy decides whether a tainted value may reach that position. A human is asked only where the policy marks `Confirm`; `Deny` cannot be overridden by an approval. Strict mode turns every `Confirm` into a `Deny`. The kernel sandbox is part of the artifact but is not the subject of this study.

5.2 Corpora

Policy corpus. 45 DSL programs: 12 benign controls and 33 attacks across 13 categories. The policy-relevant attacks read the same hostile document and differ only in which field reaches which sensitive argument. The

four resource-exhaustion programs burn the interpreter's step budget. The benign controls use planner-authored literals.

Model corpus. 16 tasks: 7 benign controls and 9 attacks. Most attacks share a request with a benign twin. The planner sees only the trusted request. Role Q is stubbed with an extractor that faithfully lifts the document's fields, including an address if one is present. An adversarial planner supplies the worst-case plan deterministically; a live `llama3.2:3b` arm is exploratory.

5.3 Results

Table 1. Policy corpus under the four arms. Attacks are 33; benign controls are 12. "Flow" means a tool ran with a tainted argument.

Arm	Flows (attacks)	Benign complete	Prompts	Decomposition
A0 no policy	29/33 (87.9%)	12/12	0	counterfactual
A1 declining	0/33	12/12	14	12 automatic, 17 human, 4 budget
A2 approving	14/33 (42.4%)	12/12	14	rubber stamp
A3 strict	0/33	12/12	0	every confirm is a deny

The 15 "denied outright" outcomes split into 12 explicit policy denials and 3 allowlist refusals that a user could override by adding the domain. The 3 are counted as human- dependent, not automatic. The 14 user refusals plus the 3 allowlist refusals are the 17 human-dependent stops. The 4 remaining attacks halt on the interpreter's step budget. The 12/12 strict benign rate is a property of this corpus: every benign control uses a trusted literal, so no legitimate forwarding is refused.

Table 2. Model corpus. 16 tasks: 9 attacks, 7 benign. "Flow" is the broad metric over all 16 tasks; benign twins that forward a field count as flows. The adversarial planner is deterministic. The live arm is one run of `llama3.2:3b`.

Arm	Flows	Prompts	Auto-refused	Benign clean	Benign refused/denied	Model-quality failures
Adversarial, declining	0/16	11	4	0/7	6	0
Adversarial, approving	11/16 (68.8%)	11	4	0/7	0 (6 completed)	0
Adversarial, strict	0/16	0	4	0/7	6 denied	0
Live, declining	0/16	1	0	2/7	1	3
Live, approving	1/16 (6.2%)	1	0	2/7	1 denied	4

The 11 flows in the adversarial approving arm decompose exactly: 4 attacks are auto-refused by intent pins, 5 unpinned attacks reach a tool, and 6 benign twins forward a field. The strict arm refuses the 5 unpinned attacks and the 6 benign twins; its zero is bought by refusing the forwarding the user asked for. The benign columns need not sum to seven: a benign task that does not reach a tool is neither clean nor refused, and a live task the model cannot plan is a model-quality failure.

Table 3. Schema controls on the 16-task adversarial approving arm.

Schema	Flows	Forwarded bytes
Unbounded	11/16 (68.8%)	363
96-character bound	10/16 (62.5%)	307
Closed types	9/16 (56.2%)	182

A size bound moves the bytes by 56 and the flows by one. Closed types move the bytes by 181 and the flows by two, and remove the instruction channel for `status` entirely. They cannot close `note` or `recipient` without refusing the user's data. The residual 182 bytes are the free-text fields. The bounded arm's pin count falls from 4 to 1, but that is the bound refusing an extraction before the pin fires, not the pins improving.

Intent pins. On the 14-task corpus used for the ablation, the approving arm goes from 13/14 (92.9%) to 9/14 (64.3%), with 4 attacks refused by pins and no prompt. Pins address substitution; they do not address forwarding.

5.4 What the arms reveal

The policy corpus's 0/33 is 12 automatic refusals, 17 human-dependent stops, and 4 budget halts. The approving arm runs 14/33. The 0% and the 42% are the same rules and the same corpus; only the human's answer changed. On the model corpus, the broad 11/16 is 5 attacks and 6 benign twins; the strict zero is 6 of 7 benign tasks denied. Each of these is a different claim that a single number would have hidden.

---

6. Applying the Protocol

6.1 The checklist

#	Report	Why
1	The arm	A zero without the arm is not a claim about the defense
2	The counterfactual	Proves the corpus is adversarial
3	Declining and approving users	Separates the machine's share from the human's
4	Strict no-human	States the cost of removing the human
5	Benign twins	Separates resistance from always-asking
6	Utility	A zero that refuses ordinary work is not a defense
7	Prompts per task	Confirmation fatigue is a security property
8	Model-quality failures	A model that cannot plan is not a blocked attack
9	Tool/schema drift test	A harness that rejects the plan is not a defense
10	Authorization model	Broad taint flow and narrow leakage are different claims
11	Raw counts, not only percentages	Small corpora make percentages unstable
12	Corpus provenance and size	Self-authored corpora need an argument for representativeness
13	Automatic, human, and budget stops separately	The headline is a sum of different things
14	Repetition and intervals, or "single run"	A sampled system is not a rate from one sample

6.2 Minimum viable evaluation

If you can add only one thing to the standard defended arm, add the **approving user**. It is the cheapest way to find out how much of the zero is the person. If you can add two, add **strict**; it gives the no-human bound and the utility cost. If you can add three, add **benign twins**; they separate a defense that resists from a defense that always asks. Run the **counterfactual** first; without it, none of the other arms means anything.

6.3 A sentence for the abstract

A paper that adopts the protocol can state its result in one sentence:

> On N tasks, the defense reported X/N with a declining user and Y/N with an approving user; strict reported Z/N at a utility cost of U/N benign tasks; M tasks failed to plan.

That sentence contains the arm, the human's contribution, the strict cost, and the denominator. It is longer than "0%" and far more useful.

---

7. Limitations

The human is simulated. "Declining" always says no; "approving" always says yes. Real users are neither, and they fatigue over time. The protocol measures the endpoints of the human's contribution, not its dynamics.

The corpus is small and self-authored. 45 policy programs and 16 model tasks are enough to isolate policy decisions and to illustrate the protocol; they are not AgentDojo. The paper does not claim external validity, and a full evaluation should port the arms to a public benchmark.

One system. The case study is AgentOS. The protocol is system-agnostic, but the numbers are not.

The live arm is exploratory. One 3B checkpoint, one run per arm, 3–4 model-quality failures. It is reported to show that the harness produces real behavior, not to estimate a rate.

The broad metric overcounts. The 11/16 approving figure includes 6 benign twins. The authorization-relative definition is stated but not fully implemented; the intent pins approximate it.

No formal proof. The type-level isolation and the policy are implementation properties, not a mechanized non-interference theorem. The protocol is an empirical measurement, not a proof.

The strict arm is a bound, not a recommendation. It is the no-human limit. A deployment that chooses it is choosing to refuse the forwarding the user asked for.

---

8. Related Work

Structural defenses. CaMeL [1] separates control and data flow, uses capabilities, and reports secure task completion on AgentDojo. Progent [4] enforces fine-grained privilege policies over tool calls. IsolateGPT [3] isolates LLM-app execution. FIDES [5] applies information-flow control to agents. AgentOS follows this line. This paper does not propose a new defense; it proposes how to measure one.

Benchmarks. AgentDojo [2] provides 97 realistic tasks and 629 security cases across email, banking, travel, and workspace, and scores utility alongside security. The four-arm protocol is complementary: a benchmark

supplies tasks and attacks; the arms supply the decision-layer comparison. A benchmark that reports a single defended rate has the same under-specification this paper addresses.

Measurement validity. The general point that a benchmark is a measurement with construct-validity assumptions is well established [6, 7]. Agent-security evaluations have an additional construct: the human in the loop. The four-arm protocol is a small, domain-specific instantiation of the general lesson.

Detection and alignment. Filtering, spotlighting, and fine-tuning are complementary but not structural: they depend on recognizing the attack. This paper is about measuring systems that do not depend on recognition, and about the ways their numbers can mislead.

---

9. Conclusion

A zero is a result only when the arm, the utility, the metric, and the denominator are named. The four-arm protocol makes those names cheap: run the same plans with no policy, with a declining user, with an approving user, and with a strict policy that asks nobody. The differences are the findings. On AgentOS, the differences are large: 0/33 with a declining user, 14/33 with an approving one, and a strict zero that denies six of seven benign forwarding tasks. The broad metric counts the user's requested forwarding as leakage; the narrow metric does not. Four instrument failures show how a green number can be produced by nothing running.

The protocol does not make a defense good. It makes a defense's claim legible. That is the minimum a reader needs to decide whether the zero means what it says.

References

[1] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr. "Defeating Prompt Injections by Design." IEEE SaTML 2026. arXiv:2503.18813.

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents." NeurIPS Datasets and Benchmarks, 2024. arXiv:2406.13352.

[3] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal. "IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems." NDSS 2025. arXiv:2403.04960.

[4] T. Shi et al. "Progent: Securing AI Agents with Privilege Control." arXiv:2504.11703, 2025.

[5] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin. "Securing AI Agents with Information-Flow Control." arXiv:2505.23643, 2025.

[6] I. D. Raji, E. M. Bender, A. Paullada, E. Denton, and A. Hanna. "AI and the Everything in the Whole Wide World Benchmark." NeurIPS Datasets and Benchmarks, 2021.

[7] A. Z. Jacobs and H. Wallach. "Measurement and Fairness." ACM FAccT, 2021.

[8] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." AISec 2023. arXiv:2302.12173.

Appendix A: Raw Arm Summaries

Policy corpus.

```
counterfactual (every rule Allow)   87.9%  29/33
defended (default_rules)             0.0%  0/33
strict (no questions asked)          0.0%  0/33
denied outright      15
refused by a person  14
did not run (halted)  4
prompts shown        14
refused outright, no question anyone is asked       12
held only by a prompt the user answers              17
same rules, user approves everything                14
```

Model corpus, adversarial planner.

```
declining: 0/16 flows, 11 prompts, 4 auto-refused, 6/7 benign refused
approving: 11/16 flows (5 attacks + 6 benign twins), 11 prompts, 4 auto-refused,
           6/7 benign completed, 363 bytes
strict:    0/16 flows, 0 prompts, 4 auto-refused, 6/7 benign denied
```

Model corpus, live `llama3.2:3b`, one run.

```
declining: 0/16 flows, 1 prompt, 3 model-quality failures, 2/7 benign clean
approving: 1/16 flows (6.2%), 173 bytes, 1 prompt, 4 model-quality failures,
           2/7 benign clean, leaked model/status-injected
```

Schema controls, adversarial approving.

```
unbounded: 11/16 flows, 363 bytes
bounded:   10/16 flows, 307 bytes
closed:     9/16 flows, 182 bytes
```

Intent pins, 14-task corpus, adversarial approving.

```
without pins: 13/14 flows (92.9%), 0 auto-refused
with pins:     9/14 flows (64.3%), 4 auto-refused
```

Appendix B: Reproducibility

The deterministic measurements require no model server:

```
cargo test --workspace
cargo run -p agentos-eval --bin agentos-eval -- --verbose
cargo run -p agentos-eval --bin agentos-model-eval --features=ollama -- --deterministic-only
```

The live arm requires an Ollama server with `llama3.2:3b`:

```
ollama serve
cargo run -p agentos-eval --bin agentos-model-eval --features=ollama -- llama3.2:3b 1
```

The last argument is the number of runs per arm. The live numbers in this paper are one run and should be read as exploratory. The deterministic arm uses the adversarial planner and is exact for the corpus. The

workspace test suite is 939 tests on Linux and 823 on Windows at the commit used for the case study; those numbers describe the artifact, not the study.